

STAT 432 — Homework 06

Neel

October 4, 2026

Setup

All analysis is done in Python. The 569 observations are randomly split (seed 432) into 70% training ($n = 398$) and 30% test ($n = 171$). The same split is used for every model. Each predictor is standardized with the **training** mean and standard deviation, and the same transformation is applied to the test data, so the test set does not influence the distance scale.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

data = pd.read_csv("data/breast-cancer.csv")
predictors = ["mean_radius", "mean_texture", "mean_smoothness"]

train, test = train_test_split(data, train_size=0.7, random_state=432)

x_bar = train[predictors].mean()
s = train[predictors].std(ddof=1)          # training mean/SD only
X_train = ((train[predictors] - x_bar) / s).to_numpy()
X_test = ((test[predictors] - x_bar) / s).to_numpy()
y_train = train["y"].to_numpy()
y_test = test["y"].to_numpy()

def knn_fit(k):
    return KNeighborsClassifier(n_neighbors=k, weights="uniform",
                                metric="euclidean",
                                algorithm="brute").fit(X_train, y_train)
```

Question 1: Compare KNN and logistic regression

Use `breast-cancer.csv`, which contains 569 cases from the Wisconsin Diagnostic Breast Cancer dataset. Predict y (1 for malignant and 0 for benign) using `mean_radius`, `mean_texture`, and `mean_smoothness`.

Set a random seed and randomly split the observations into 70% training and 30% test data. Use the same split for every model. Standardize the predictors using the training

means and standard deviations, and apply the same transformations to the test data.

Fit ordinary logistic regression with an intercept and no penalty, and KNN with Euclidean distance, equal neighbor weights, and $k \in \{5, 15, 25, 35, 45\}$.

Using the estimated probabilities for $y = 1$ on the test data, plot all six ROC curves in one figure and report their AUC values. Which of the six fitted models do you prefer? Give a brief justification based on your results.

Solution

For KNN, the estimated probability of malignancy is the proportion of malignant cases among the k nearest training observations,

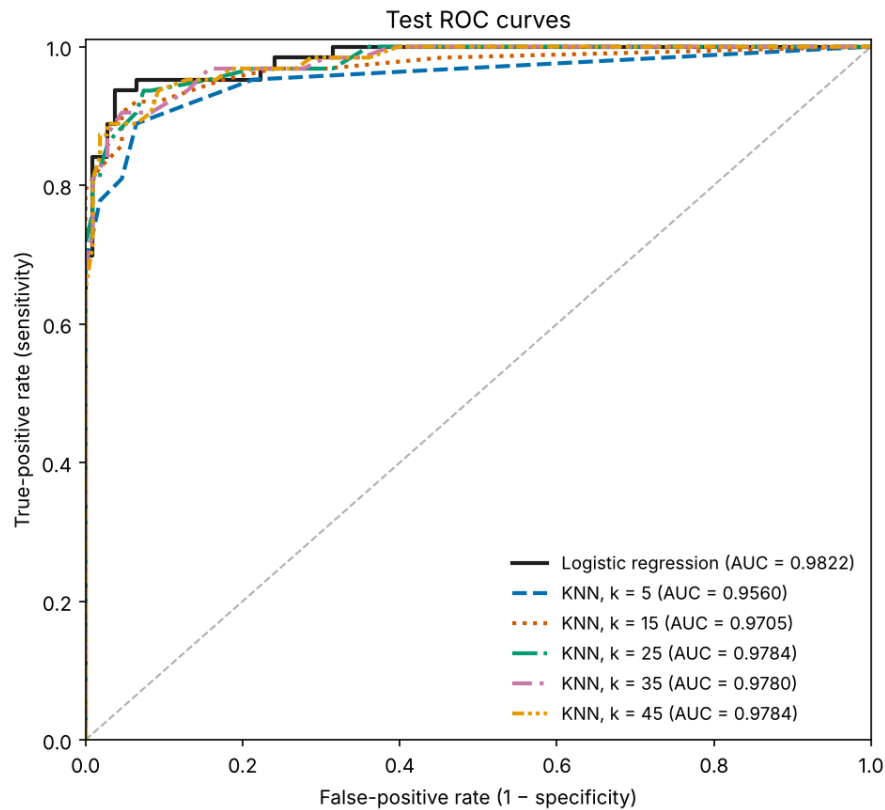
$$\hat{p}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i,$$

which is what `predict_proba` returns with uniform weights. Logistic regression is fit with an intercept and no penalty (`C=np.inf`).

```
logit = LogisticRegression(C=np.inf, fit_intercept=True, max_iter=1000)
logit.fit(X_train, y_train)

k_values = [5, 15, 25, 35, 45]
names = ["Logistic regression"] + [f"KNN, k = {k}" for k in k_values]
p_test = [logit.predict_proba(X_test)[:, 1]] + \
    [knn_fit(k).predict_proba(X_test)[:, 1] for k in k_values]
auc1 = [roc_auc_score(y_test, p) for p in p_test]

colors = ["#222222", "#0072B2", "#D55E00", "#009E73", "#CC79A7", "#E69F00"]
styles = ["-", "--", ":", "-.", (0, (5, 2, 1, 2)), (0, (3, 1, 1, 1, 1, 1))]
fig, ax = plt.subplots(figsize=(6.5, 6))
ax.plot([0, 1], [0, 1], color="0.7", linestyle="--", linewidth=1)
for p, name, a, c, ls in zip(p_test, names, auc1, colors, styles):
    fpr, tpr, _ = roc_curve(y_test, p)
    ax.plot(fpr, tpr, color=c, linestyle=ls, linewidth=2,
            label=f"{name} (AUC = {a:.4f})")
ax.set(xlim=(0, 1), ylim=(0, 1.01),
       xlabel="False-positive rate (1 - specificity)",
       ylabel="True-positive rate (sensitivity)", title="Test ROC curves")
ax.legend(loc="lower right", frameon=False, fontsize=9)
plt.show()
```



Test ROC curves for logistic regression and five KNN fits on the same 171 test observations.

Model	Test AUC
Logistic regression	0.9822
KNN, $k = 5$	0.9560
KNN, $k = 15$	0.9705
KNN, $k = 25$	0.9784
KNN, $k = 35$	0.9780
KNN, $k = 45$	0.9784

Preferred model: logistic regression. It has the highest test AUC (0.9822), so it ranks malignant test cases above benign ones most effectively of the six models. It is also the simplest and most interpretable: three coefficients and an intercept, rather than storing the full training set. KNN improves as k grows from 5 to 25 (0.956 to 0.978) and then levels off near 0.978, still slightly below logistic regression. This suggests that a smooth, roughly linear boundary in these three predictors works well, and that small neighborhoods add variance without capturing useful local structure.

This comparison uses a single test split, and the differences among the top models are small, so they do not show that one method is always better. AUC also measures ranking, not accuracy at a particular cutoff. A more careful model choice would select the method and k by cross-validation on the training data.

Question 2: Training and test ROC curves

Use the same data, training/test split, and standardization as in Question 1. Fit KNN with Euclidean distance, equal neighbor weights, and $k \in \{1, 3, 5, 10, 20\}$.

Use each fitted model to estimate probabilities for both the training and test observations. For training predictions, include each observation itself among its k nearest neighbors.

For each k , use a separate panel to show the training and test ROC curves together. Report both AUC values.

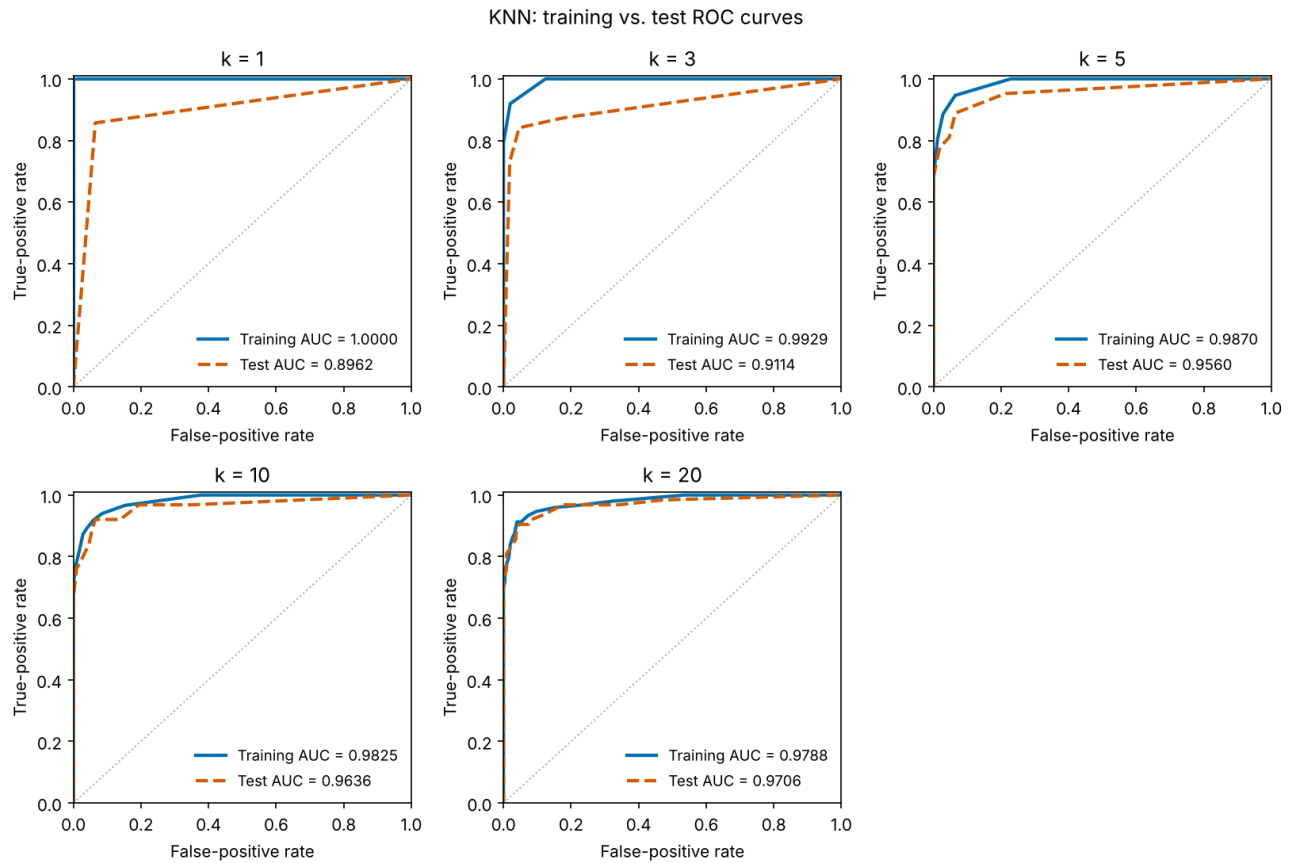
Explain the difference between training and test performance and how it changes with k . In particular, explain what happens at $k = 1$ and why training performance may not reflect performance on new observations.

Solution

Passing `X_train` to `predict_proba` makes each training observation one of its own k nearest neighbors (at distance 0), as the question requires.

```
k_values2 = [1, 3, 5, 10, 20]
rows = []
fig, axes = plt.subplots(2, 3, figsize=(11, 7.5))
for ax, k in zip(axes.flat, k_values2):
    fit = knn_fit(k)
    p_tr = fit.predict_proba(X_train)[: , 1] # each point is its own neighbor
    p_te = fit.predict_proba(X_test)[: , 1]
    a_tr, a_te = roc_auc_score(y_train, p_tr), roc_auc_score(y_test, p_te)
    rows.append((k, a_tr, a_te, a_tr - a_te))
    for y, p, a, lab, c, ls in [(y_train, p_tr, a_tr, "Training", "#0072B2", "-"),
                                (y_test, p_te, a_te, "Test", "#D55E00", "--")]:
        fpr, tpr, _ = roc_curve(y, p)
        ax.plot(fpr, tpr, color=c, linestyle=ls, linewidth=2,
                label=f"{lab} AUC = {a:.4f}")
    ax.plot([0, 1], [0, 1], color="0.7", linestyle=":", linewidth=1)
    ax.set(title=f"k = {k}", xlim=(0, 1), ylim=(0, 1.01),
           xlabel="False-positive rate", ylabel="True-positive rate")
    ax.legend(loc="lower right", frameon=False, fontsize=9)
axes.flat[-1].set_visible(False)
fig.suptitle("KNN: training vs. test ROC curves")
fig.tight_layout()
plt.show()

print(pd.DataFrame(rows, columns=["k", "Training AUC", "Test AUC", "Gap"])
      .round(4).to_string(index=False))
```



Training (solid) and test (dashed) ROC curves for each neighborhood size. Each training observation is included among its own neighbors.

k	Training AUC	Test AUC	Gap
1	1.0000	0.8962	0.1038
3	0.9929	0.9114	0.0814
5	0.9870	0.9560	0.0310
10	0.9825	0.9636	0.0189
20	0.9788	0.9706	0.0082

Training vs. test performance. For every k , training AUC is higher than test AUC because each training observation's own label is used to predict it. As k increases, training AUC falls slightly (1.000 to 0.979), test AUC rises (0.896 to 0.971), and the gap shrinks from 0.104 to 0.008. A larger k averages over more neighbors, so a single label, including the observation's own, has less influence. The estimated probabilities have lower variance and the fit overfits less. If k grew much larger, averaging over distant observations would add bias and miss local structure, and test performance would eventually get worse. Neither curve has to change monotonically on a particular split.

What happens at $k = 1$. Each training observation is its own nearest neighbor, so its estimated probability equals its own label: $\hat{p}(\mathbf{x}_i) = y_i \in \{0, 1\}$. Every malignant case gets

probability 1 and every benign case gets 0, which separates the classes perfectly and gives a training AUC of exactly 1. This is memorization, not predictive ability. A new observation's label is not in the training set, so its prediction copies the label of a *different* nearby point, which may be noisy or on the wrong side of the boundary. Test AUC is only 0.896. Because 1-NN probabilities are only 0 or 1, the test ROC curve has a single nontrivial corner point, and its AUC equals $(\text{sensitivity} + \text{specificity}) / 2$.

So training performance, especially with small k , is an overly optimistic estimate of performance on new observations. It reuses the labels it is being evaluated on. Held-out data (a test set or cross-validation) is needed to estimate how the model will predict new cases.

Question 3: Peer evaluation of Homework 5

Posted GitHub Issue: <https://github.com/nupurkalele1/stat432-fall2026/issues/3>

Reviewed report: <https://github.com/nupurkalele1/stat432-fall2026/blob/main/homework/week-05/homework-05.pdf>